

Trevor Streng

6/15/2025

Assignment 2

Part 1

We first started by converting the image to grayscale. We then used different numpy arrays containing positive and negative numbers to emphasize vertical, horizontal, and diagonal lines. This worked by predefining a kernel of numbers and multiplying that kernel across all pixels. After detecting edges, we used a similar approach with different sized kernels filled with all ones to blur the image. We used 2x2, 3x3, and 5x5 kernels. The 5x5 kernel causes the most blur, and the 2x2 kernel causes the least blur. This can help in a convolutional neural network to ensure some of the noise is removed from the picture and the network doesn't learn irrelevant information. Next, we used the Sobel operator which is another way to do edge detecting. This seemed to define the horizontal and vertical lines slightly more. Next, I used a similar approach to perform corner detection. The difference being the zeros and ones in the kernels were in the shape of corners rather than lines. Finally, I performed some of these on an image I found online of a container ship. I started by blurring the image and then detecting vertical, horizontal, and diagonal lines. I blurred it using a 3x3 array and used the original kernels for edge detection.

Part 2

For my Convolutional Neural Network architecture, I started with 4 convolutional layers. The first layer took in the 3 RGB values and output 10 values with no padding and a stride of 1. The second layer took the 10 layers in and output 15 layers. I used a 5x5 kernel with 2 padding and kept the stride at 1. The next layer was similar but took in 15 values and output 25. The last layer I took in the 25 values and output 15 while using a 3x3 kernel and 1 padding. I seemed to have the best results using maxPooling on layers one, two, and four. I couldn't use them on all 4 layers, otherwise I would get a shape error. After the convolutional layers, I used view to flatten it into a 1-dimensional matrix. I figured out that starting with 32x32, then plugging in into the first convolutional layer with no padding, it reduced it to 30x30. Then, using maxPool2d(2, 2) reduced it to 15x15. On the next layer I included padding so nothing was removed there, but I maxPooled again to reduce it to 7x7. On the third layer, nothing should have changed in the direction, but on the fourth layer, I again used maxPooling, reducing it to a 3x3. The final layer's output parameter was 20, this gave us a shape of 3x3x20. This is how I found the number needed to flatten the matrix, and the same number was used to determine how many inputs on the first hidden layer. I had two hidden layers, both Linear. The first hidden layer also used ReLU activation.

My findings with the SGD optimizer with a learning rate of 0.01 and 10 epochs provided me with a training loss of 1.150113 and a validation loss of 1.096916. After testing the network, I had a test loss of 1.108566 and a test accuracy of 60 % (6089/10000). My highest scoring category was ships at 75% accuracy, and my lowest two were birds and cats, both at 42% accuracy. In my comparison, using the ADAM optimizer, it seemed to find the least loss very early on and did not require many epochs. My total time for training was 3 minutes and 8.1 seconds with the SGD optimizer and a similar time with the ADAM optimizer.

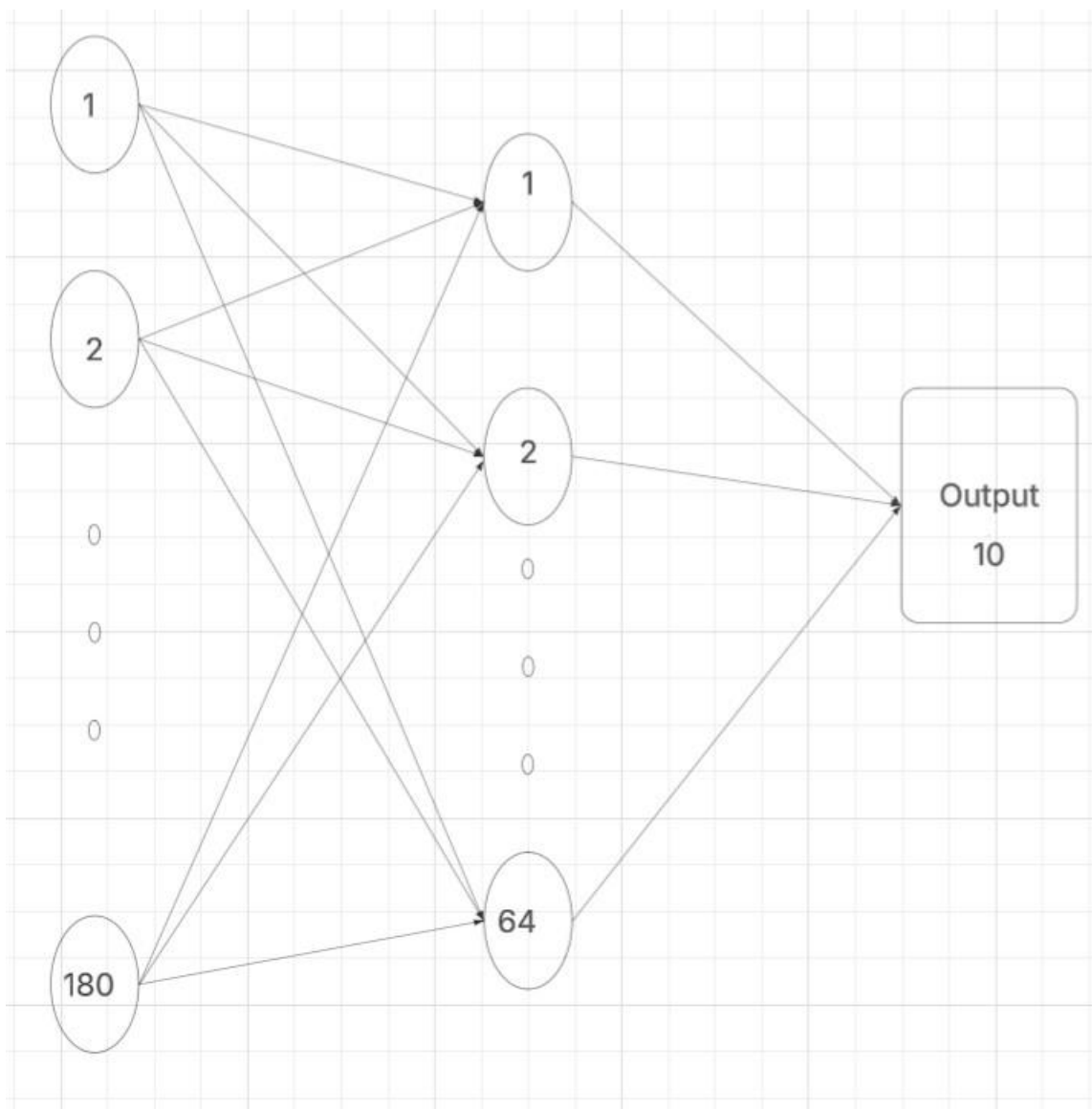
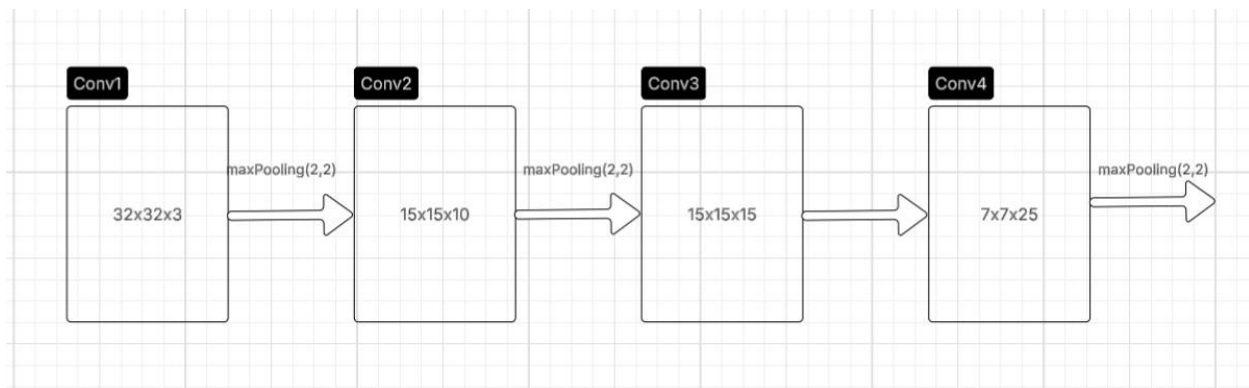
I want to keep messing with the network to try and get better results. I noticed the convolutional layers can help extract information when done in a certain way. What I noticed while experimenting was that adding more layers sometimes helped. It also seemed that having the middle layer not maxPool allowed the network to extract things earlier on decreasing the loss. But maxPooling too late can cause overfitting.

- Accuracy: 60%
- Time taken for training: 3 minutes 8.1 seconds
 - Trained on GPU
- Number of parameters:
 - Convolutional layers
 - $(3 \times 3 \times 3 + 1) \times 10 = 280$
 - $(10 \times 5 \times 5 + 1) \times 15 = 3765$
 - $(15 \times 5 \times 5 + 1) \times 25 = 9400$
 - $(25 \times 3 \times 3 + 1) \times 20 = 4520$
 - Flattening
 - $3 \times 3 \times 20 \Rightarrow 180$
 - Fully Connected
 - $(180 + 1) \times 64 = 11584$
 - $(64 + 1) \times 10 = 650$
 - Total: 30,199
 - What helps what does not?

What seemed to help was max pooling during certain layers, expanding the parameters and then reducing them before fully connected layers. What didn't seem to help was too small of kernels, not enough convolutional layers, and too many parameters in the convolutional layers.

Github link:

https://github.com/TrevorStreng/CSCI611_Summer25_Trevor_Streng/tree/main/assignment_2



... Test Loss: 1.108566

Test Accuracy of airplane: 70% (701/1000)
Test Accuracy of automobile: 82% (823/1000)
Test Accuracy of bird: 42% (423/1000)
Test Accuracy of cat: 42% (420/1000)
Test Accuracy of deer: 43% (435/1000)
Test Accuracy of dog: 52% (525/1000)
Test Accuracy of frog: 70% (708/1000)
Test Accuracy of horse: 65% (658/1000)
Test Accuracy of ship: 75% (754/1000)
Test Accuracy of truck: 64% (642/1000)

Test Accuracy (Overall): 60% (6089/10000)